TransitOps

Smart Transport Operations Platform

Post-Phase-1 Development Report

Phases 2 – 5 | RBAC Enforcement | UI/UX Upgrades

July 2026 · Odoo Virtual Hackathon Round

Table of Contents

Phase 2 — Vehicle & Driver Management

Phase 2 delivers complete CRUD operations for the two core fleet entities: **Vehicles** and **Drivers**. Each entity follows the same three-layer architecture: Pydantic schema → service → Streamlit page.

2.1 New Files Created

File	Purpose
app/schemas/vehicle_schema.py	VehicleCreate, VehicleUpdate — field validation, type coercion
app/schemas/driver_schema.py	DriverCreate, DriverUpdate — license, score, expiry validation
app/services/vehicle_service.py	get_all, get_by_id, create, update, delete, fleet_summary
app/services/driver_service.py	get_all, get_by_id, create, update, delete, driver_summary
frontend/pages/page_fleet.py	KPI bar, filter table, add/edit forms, status change, delete
frontend/pages/page_drivers.py	KPI bar, license alerts, filter, add/edit, status, delete

2.2 Business Rules

- Registration number must be unique — service raises ValueError on duplicate.
- Safety score < 70 → Driver auto-suspended on create/update.
- Vehicles with trip history are Retired (not hard-deleted); drivers set to Off Duty.
- Driver license expiry < today → flagged as EXPIRED in table + red alert banner.
- Driver license expiry ≤ 30 days → flagged as EXPIRING SOON + yellow warning banner.
- Cannot change vehicle status from 'On Trip' manually while a trip is Dispatched.

2.3 Key UI Features

Feature	Details
6-KPI header (Drivers)	Total, Available, On Trip, Suspended, Expired License, Expiring Soon
5-KPI header (Fleet)	Total, Available, On Trip, In Maintenance, Retired
Filter bar	Type + Status (Fleet); Status + Category + Name search (Drivers)
Color-coded table	Status column styled green/amber/red/gray via Pandas Styler
Add tab / Actions tab	Tabs shown/hidden dynamically based on RBAC can() check
Selection card	Clicking a vehicle/driver shows a styled detail card, then action buttons

Phase 3 — Trip Dispatch Engine

Phase 3 implements the core operational workflow: create a trip, validate all business rules, dispatch it (locking vehicle + driver), complete it (releasing resources), or cancel it.

3.1 Trip Lifecycle

Status	Trigger	Vehicle	Driver
Draft	Trip created	Available	Available
Dispatched	Dispatch action	On Trip	On Trip
Completed	Complete action	Available	Available
Cancelled	Cancel action	Available*	Available*

** Vehicle and Driver only released if trip was Dispatched at time of cancellation.*

3.2 Dispatch Validation Rules

- Vehicle status must be **Available** at dispatch time.
- Driver status must be **Available** at dispatch time.
- Driver license must NOT be expired (hard block, not a warning).
- cargo_weight_kg must be ≤ vehicle.max_capacity_kg.
- planned_distance_km must be > 0; revenue and cargo_weight must be ≥ 0.
- Soft capacity check also runs at trip creation to catch obvious errors early.
- Trip code auto-generated: **TRP-DDMMYY-XXXXXX** (date + 6-char UUID fragment).

3.3 New Files

File	Purpose
app/schemas/trip_schema.py	TripCreate, TripUpdate, TripComplete with validators
app/services/trip_service.py	Full lifecycle + get_available_vehicles/drivers helpers
frontend/pages/page_trips.py	6-KPI header, All Trips tab, Create Trip tab, Actions panel

Phase 4 — Maintenance & Fuel/Expense Logging

4.1 Maintenance Workflow

- Vehicle must NOT be On Trip before logging maintenance (cannot repair a moving truck).
- Vehicle must NOT already be In Shop (must close existing record first).
- Logging maintenance → Vehicle status: **Available** → **In Shop**.
- Closing maintenance → Vehicle status: **In Shop** → **Available**.
- Only one Active maintenance record allowed per vehicle at a time.
- Closing allows optional actual_cost override (recorded against the log).

4.2 Fuel & Expense Logging

Entity	Key Fields	Validation
FuelLog	vehicle, liters, cost, date, optional trip_id	liters > 0; cost > 0; trip must belong to same vehicle if linked
Expense	vehicle, category, amount, date	category ∈ [Toll, Insurance, Fine, Parking, Loading/Unloading, Drive

4.3 New Files

File	Purpose
app/schemas/maintenance_schema.py	MaintenanceCreate, MaintenanceClose
app/schemas/fuel_schema.py	FuelCreate, ExpenseCreate + EXPENSE_CATEGORIES list
app/services/maintenance_service.py	log, close, summary, vehicle selector helpers
app/services/fuel_service.py	add_fuel_log, add_expense, cost_summary, get_all
frontend/pages/page_maintenance.py	4-KPI header, Log tab, All Records tab, Close panel
frontend/pages/page_fuel_expenses.py	4-tab layout: Log Fuel, Log Expense, Fuel Records, Expense Records

Phase 5 — Analytics & Settings

5.1 Analytics Page

Chart	Type	Data Source
Revenue by Vehicle	Bar (green)	Trip.revenue grouped by vehicle (Completed only)
Completed Trips by Month	Bar (blue)	Trip.created_at grouped by month
Fuel Efficiency per Vehicle	Bar (color-coded)	trip.fuel_consumed_l / planned_distance_km
Cost Breakdown	Donut pie	FuelLog.cost + Expense.amount + Maintenance.cost
Total Trips per Vehicle	Bar (purple)	All trips (any status) by vehicle

All charts are rendered with **Plotly** using the app's dark theme palette. CSV export (3 buttons: Trips, Fuel, Expenses) is gated to Fleet Manager and Financial Analyst roles.

5.2 Settings Page

- System Info tab — App name, DB type, framework, ORM, debug mode.
- User Management tab — Role-colour-coded user table + role change form (Fleet Manager only).
- RBAC Matrix tab — Read-only colour-coded view of all role permissions.
- Danger Zone — 'Reset Database' button that calls reset_database() (dev tool, FM only).
- Access gated: renders an error and stops for any non-Fleet-Manager role.

RBAC Enforcement — Two-Layer System

6.1 Architecture

Two independent layers prevent unauthorised access:

Layer 1 — Coarse (Sidebar)
PERMISSIONS dict in rbac.py. Maps role → module → 'none'/'view'/'edit'. Sidebar only renders items where access ≠ 'none'. Settings hidden for all non-FM roles.

Layer 2 — Granular (In-page)
ROLE_ACTIONS dict in rbac.py. Maps role → action → bool. e.g. 'fleet.add', 'trips.dispatch', 'fuel.add'. Pages call can(action) from auth_guard.py to show/hide individual buttons and forms.

6.2 RBAC Matrix

Module	Fleet Manager	Dispatcher	Safety Officer	Financial Analyst
Dashboard	■ View	■ View	■ View	■ View
Fleet	═■ Full CRUD	■ View only	■ View only	■ View only
Drivers	═■ Full CRUD	■ View only	═■ Add/Edit/Status (no del)	■ View only
Trips	═■ Full workflow	═■ Full workflow	■ View only	■ View only
Maintenance	═■ Full CRUD	■ View only	■ View only	■ View only
Fuel & Expenses	═■ Full CRUD	■ View only	■ View only	═■ Add/View
Analytics	■ + CSV Export	■ Charts only	■ Charts only	■ + CSV Export
Settings	═■ Full	■ Hidden	■ Hidden	■ Hidden

6.3 How It Works in Code

```
# auth_guard.py — used in every page
from frontend.components.auth_guard import can

if can('fleet.add'):
show_add_vehicle_tab()

if can('trips.dispatch') and trip['status'] == 'Draft':
show_dispatch_button()
```

UI/UX Upgrades

7.1 Dashboard Fixes

- **Removed dead filter row** — Type / Status / Region dropdowns had no effect and confused users.
- **Recent Trips now live** — Added Vehicle and Driver columns; queries live DB on every render.
- **Flat bar → Donut chart** — Fleet status donut with center label (total count) + mini stat row.
- **8 KPIs instead of 7** — Added Total Revenue tile (Rs. from completed trips).

7.2 Selection Card (Actions Panel)

All three management pages (Fleet, Drivers, Trips) previously showed a cramped 3-column layout where the status dropdown was squeezed between Edit and Delete buttons. This was replaced with a **styled selection card** that shows all key details of the chosen item, followed by clearly labelled action buttons and a separate 'Change Status' row.

- Orange left-border card with flex-wrap meta tags (Model, Type, Capacity, etc.).
- Colour-coded status badge inside card (green/amber/red/gray).
- Driver card shows safety score in colour (green ≥ 80, amber ≥ 70, red < 70) + license badge.
- Trip card shows route, vehicle, driver, cargo weight, revenue, status badge.
- Action buttons only appear for the actions actually available for the selected item's status.
- Status change moved to its own labelled section with a [3:1] wide selector + Apply button.

7.3 CSS Enhancements (style.css)

- Primary button — gradient fill, glow box-shadow, translateY hover lift.
- Secondary button — border turns orange on hover.
- Form inputs — border highlights orange on focus with soft glow ring.
- Dataframe — border-radius 10px on all tables.
- Custom scrollbar — thin 6px dark scrollbar matching theme.
- Tab transitions — subtle radius on tab list items.
- .selection-card, .sc-badge-* — new classes powering the actions panel.

Errors Encountered & Resolutions

Error 1: StreamlitAPIException: st.switch_page() page not found

Cause: Phase 1 initially used st.switch_page() with SDD file paths. Streamlit only supports pages/ folder convention.

Fix: Abandoned st.switch_page(); built custom PAGE_MAP router in frontend/router.py. Single app.py entry point with session_state['page'] driving all navigation.

Error 2: UnicodeEncodeError: 'cp1252' codec can't encode character

Cause: Windows terminal (cp1252) can't print Unicode emoji from Python logging on stdout.

Fix: Added sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8', errors='replace') at top of all CLI scripts (smoke tests, seed, PDF generator).

Error 3: SQLite OperationalError: database is locked

Cause: Smoke tests tried to reset_database() (delete .db file) while Streamlit had an open connection.

Fix: Always stop Streamlit before running smoke tests. reset_database() now has a clear error message. Documented in dev workflow.

Error 4: SyntaxError: invalid syntax (walrus := operator in assert)

Cause: Smoke test p345 used 'assert f.cost_per_l_calc := round(...)' — walrus in assert is invalid Python.

Fix: Removed the malformed assert; the cost_per_l field is computed in the service, not the schema.

Error 5: Pydantic ValidationError: 'model_post_init' not called for default date

Cause: MaintenanceCreate used 'date: datetime.date = None' but Pydantic v2 doesn't call model_post_init on None defaults the same way.

Fix: Changed to use model_post_init with object.__setattr__ for the default date assignment, matching the pattern used in FuelCreate and ExpenseCreate.

Error 6: Protobuf 'Descriptors cannot be created directly' on page imports

Cause: Importing Streamlit pages (which import streamlit) outside the Streamlit runtime raises a protobuf version conflict error.

Fix: This is a known Streamlit limitation. Backend-only smoke tests skip page imports. Pages are verified by running the app manually. Not a code error.

Error 7: Dashboard Recent Trips showing stale/no data

Cause: The recent trips query existed but the DataFrame lacked Vehicle and Driver columns. Also, the page had a non-functional filter row (Type / Status / Region) that misled users.

Fix: Removed filter row. Added v_map and d_map lookup in _get_dashboard_data(). Recent trips now shows Trip Code, Route, Vehicle, Driver, Revenue, Status.

Error 8: Actions panel UX — cramped 3-column layout

Cause: Vehicle/Driver/Trip action panels had Edit | [Status dropdown + Apply] | Delete in three equal columns, making the status dropdown tiny and confusing.

Fix: Replaced with a styled .selection-card HTML component showing item details, followed by clean action buttons (dynamic count) and a dedicated status-change row.

8.1 Directory Structure

```
TransitOps/
■■■ app.py ← Single entry point; session router
■■■ .env ← DATABASE_URL, SECRET_KEY, APP_NAME, DEBUG
■■■ app/
■ ■■■ config.py ← Loads .env via python-dotenv
■ ■■■ constants.py ← Enums (VehicleStatus, TripStatus, …)
■ ■■■ logger.py ← get_logger() factory (rotating file handler)
■ ■■■ auth/
■ ■ ■■■ rbac.py ← PERMISSIONS + ROLE_ACTIONS + helpers
■ ■ ■■■ security.py ← bcrypt hash/verify
■ ■■■ database/
■ ■ ■■■ engine.py ← get_session(), init_db(), reset_database()
■ ■ ■■■ seed.py ← 4 roles, 4 users, 6 vehicles, 5 drivers, 5 trips
■ ■■■ models/ ← 8 SQLAlchemy ORM models
■ ■ ■■■ (base, user, role, vehicle, driver, trip, maintenance, fuel_log, expense)
■ ■■■ schemas/ ← Pydantic v2 input/update schemas
■ ■ ■■■ (vehicle, driver, trip, maintenance, fuel)
■ ■■■ services/ ← Business logic, zero Streamlit imports
■ ■■■ (auth, vehicle, driver, trip, maintenance, fuel)
■■■ frontend/
■ ■■■ router.py ← PAGE_MAP dict; maps page names to render()
■ ■■■ assets/style.css ← Global dark theme CSS
■ ■■■ components/
■ ■ ■■■ auth_guard.py ← require_auth(), require_role(), can(action)
■ ■ ■■■ kpi_card.py ← render_kpi_card() HTML component
■ ■ ■■■ sidebar.py ← Role-filtered navigation sidebar
■ ■■■ pages/ ← 8 page modules each with render()
■ ■■■ (dashboard, fleet, drivers, trips, maintenance,
■ fuel_expenses, analytics, settings)
■■■ data/transitops.db ← SQLite database
■■■ docs/ ← Generated PDF reports
```

8.2 Key Design Decisions

Zero-Streamlit backend — app/ contains no streamlit imports. All business logic is pure Python. Pages in frontend/ are the only place st.* is called.

Custom router over st.switch_page — Streamlit's native multi-page routing requires files in pages/ and breaks with SDD folder structure. PAGE_MAP in router.py gives full control.

SQLite + SQLAlchemy ORM — SQLite requires no server for hackathon. SQLAlchemy ORM with mapped_column and typed Mapped[] fields. reset_database() available for development iterations.

Pydantic v2 schemas — All user input validated before reaching services. ValidationError messages shown directly in UI via st.error(). Services only receive clean data.

bcrypt authentication — Passwords hashed with bcrypt at seed time. Login validates via bcrypt.checkpw(). JWT/sessions simulated via Streamlit session_state.